



Module 4. Recurrent Neural Networks

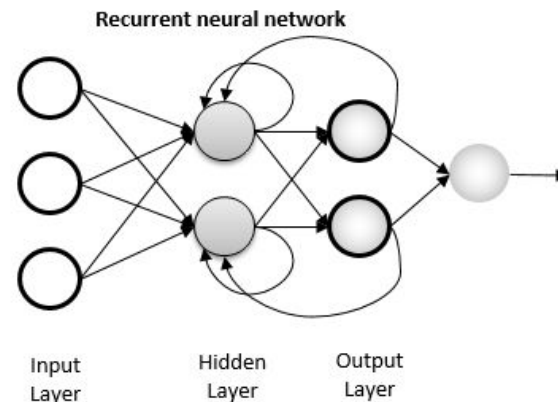
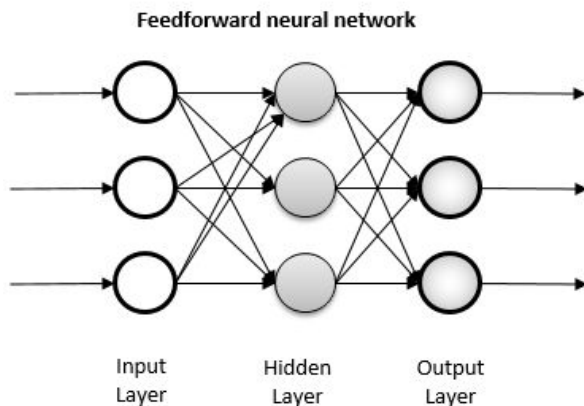
Recurrent Neural Networks (RNNs)
Long Short Term Memory (LSTMs)
Gated Recurrent Units (GRUs)



Conceptual Introduction

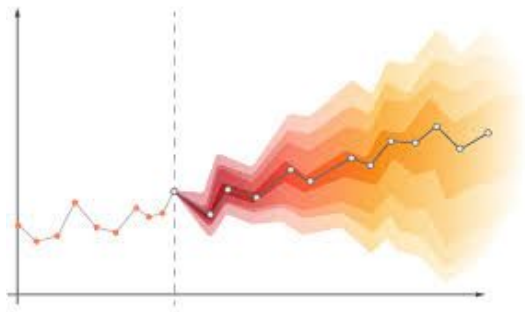
What Makes RNNs Different from Feedforward Networks?

- Introduced in the 1980s by researchers David Rumelhart, Geoffrey Hinton, and Ronald J. Williams
- Feedforward networks process inputs independently – no memory.
- RNNs retain a hidden state that carries information across time steps - memory.
- Designed for sequential data, where past inputs influence the present.



Real-World Applications of RNNs

- **Time Series Forecasting:** Stock prices, weather prediction.
- **Natural Language Processing:** Text generation, sentiment analysis.
- **Speech Recognition:** Converting speech to text.
- **Autonomous Systems:** Handwriting generation, robotics.



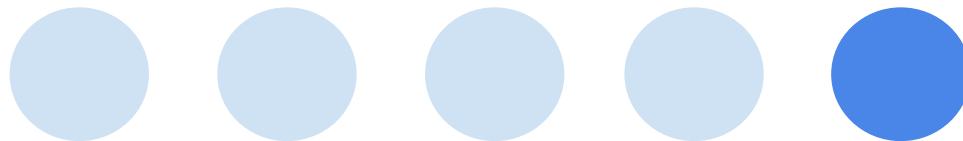
The Idea of Recurrence

- RNNs maintain a hidden state that acts as memory.
- Each step's output depends on the **current input** + **past hidden** state.
- Enables learning sequential dependencies over time.
- **Analogy**: Can you predict the direction in which the dot is going to move?



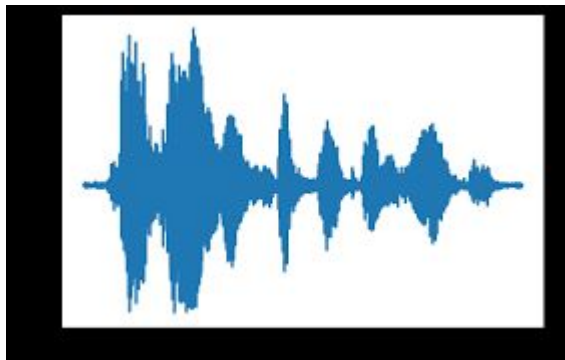
The Idea of Recurrence

- RNNs maintain a hidden state that acts as memory.
- Each step's output depends on the **current input** + **past hidden** state.
- Enables learning sequential dependencies over time.
- **Analogy:** Understanding the sequence of the data we can predict is moving to the right



Sequence Data

Sequence data comes in many forms



- Audio

Sequence
Data
Comes
In
Many
Forms

- Text



Understanding the Mechanics



Example

Can you tell the first letters of the alphabet?

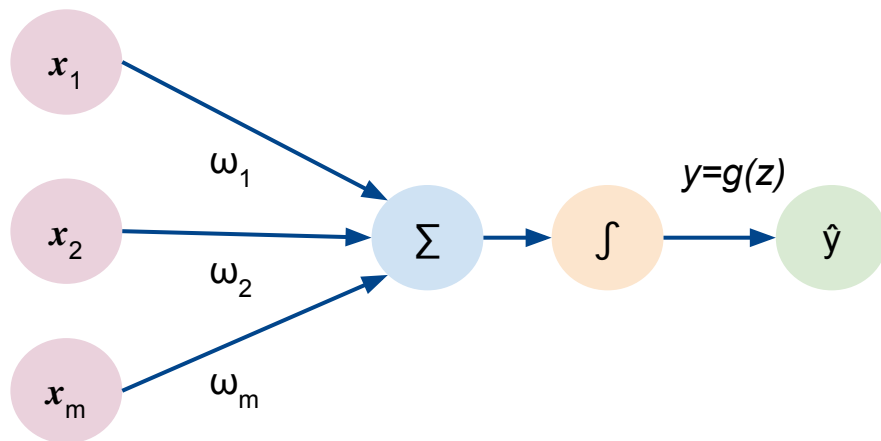
A B C D E F G

Now backwards

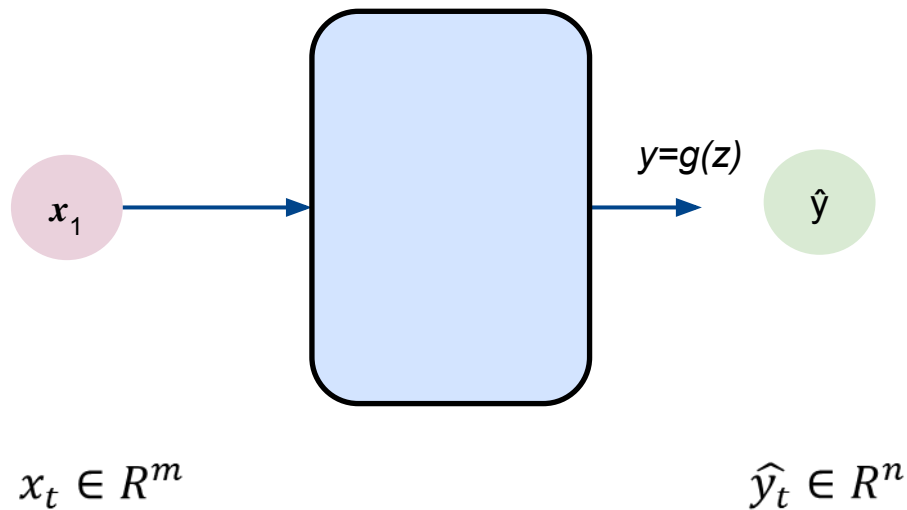
G F E D C B A

Harder!

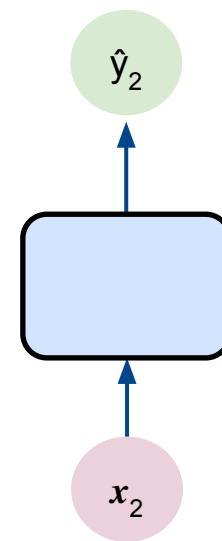
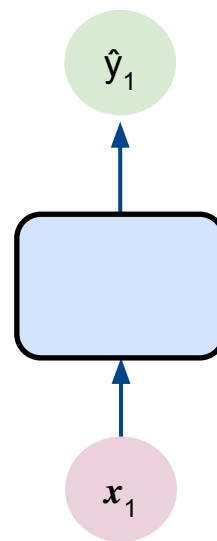
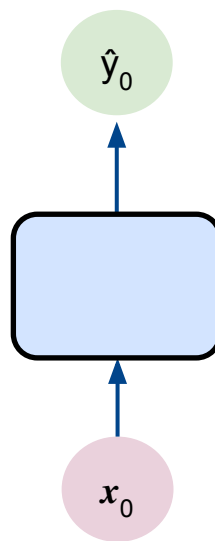
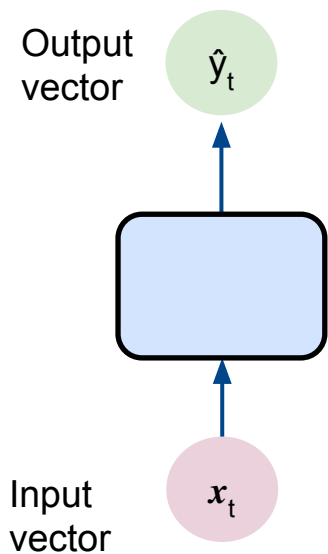
Recap - Perceptron



Recap – Feed Forward Networks

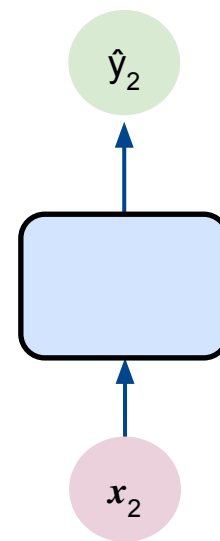
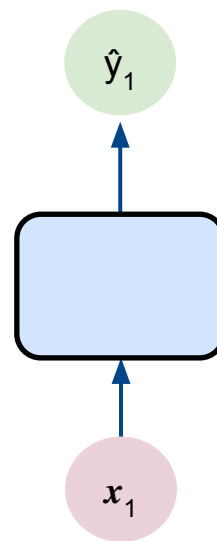
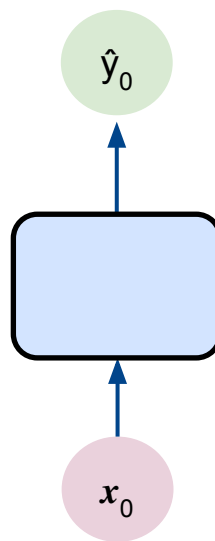
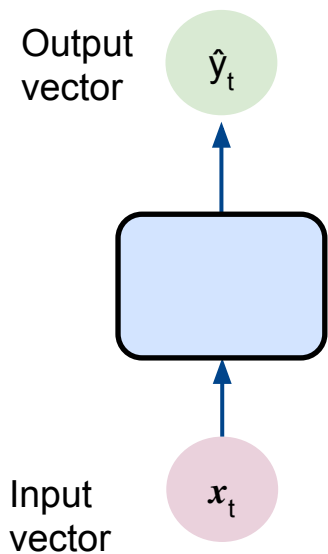


Individual Time Steps



$$\hat{y}_t = f(x_t)$$

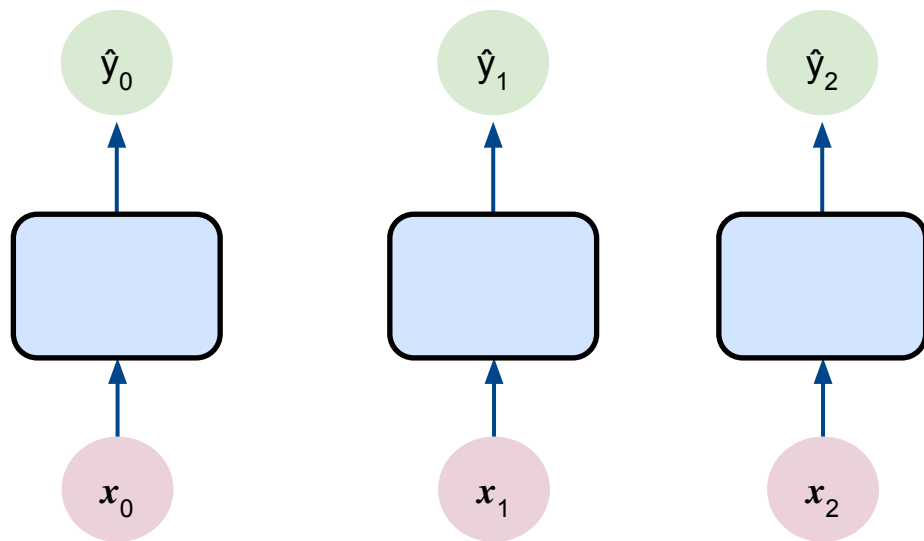
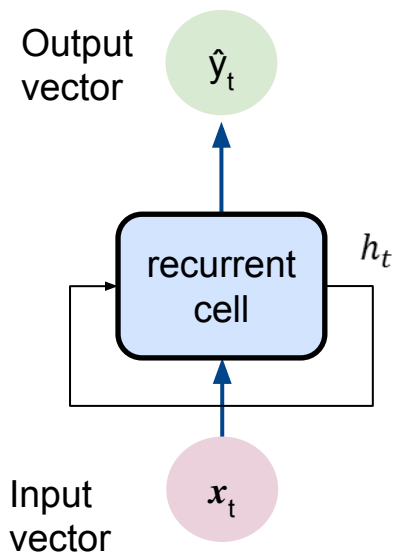
Neurons with Recurrence



$$\hat{y}_t = f(\underbrace{x_t}_{\text{input}}, \underbrace{h_{t-1}}_{\text{past memory}})$$

output input past memory

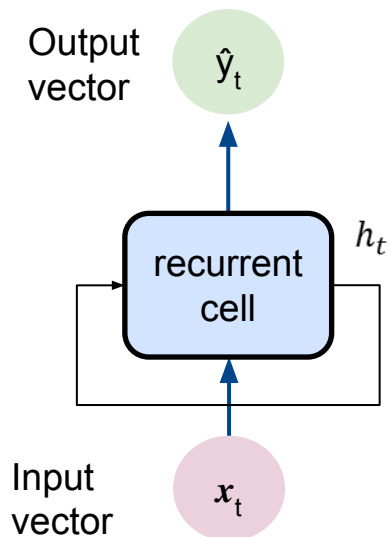
Neurons with Recurrence



$$\hat{y}_t = f(\underbrace{x_t}_{\text{input}}, \underbrace{h_{t-1}}_{\text{past memory}})$$

output input past memory

Neurons with Recurrence



$$h_t = f_W(x_t, h_{t-1})$$

Cell state

Function
with
weights W

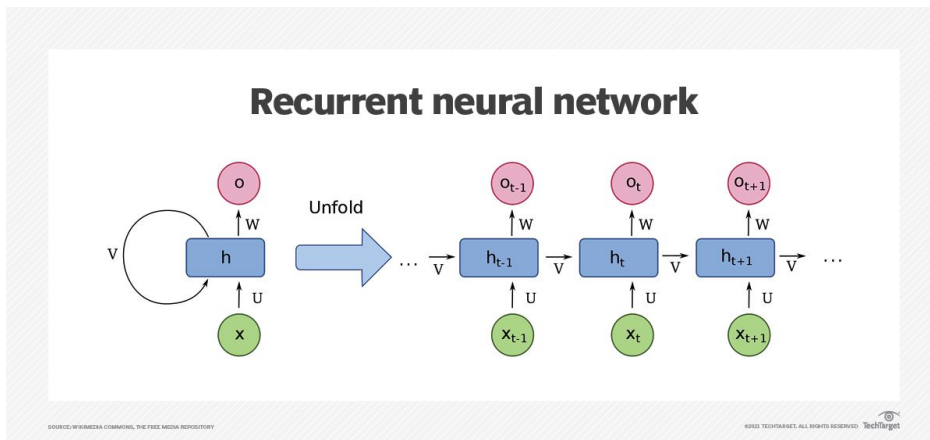
Input

Old state

RNN Architecture - Unrolling/Unfolding

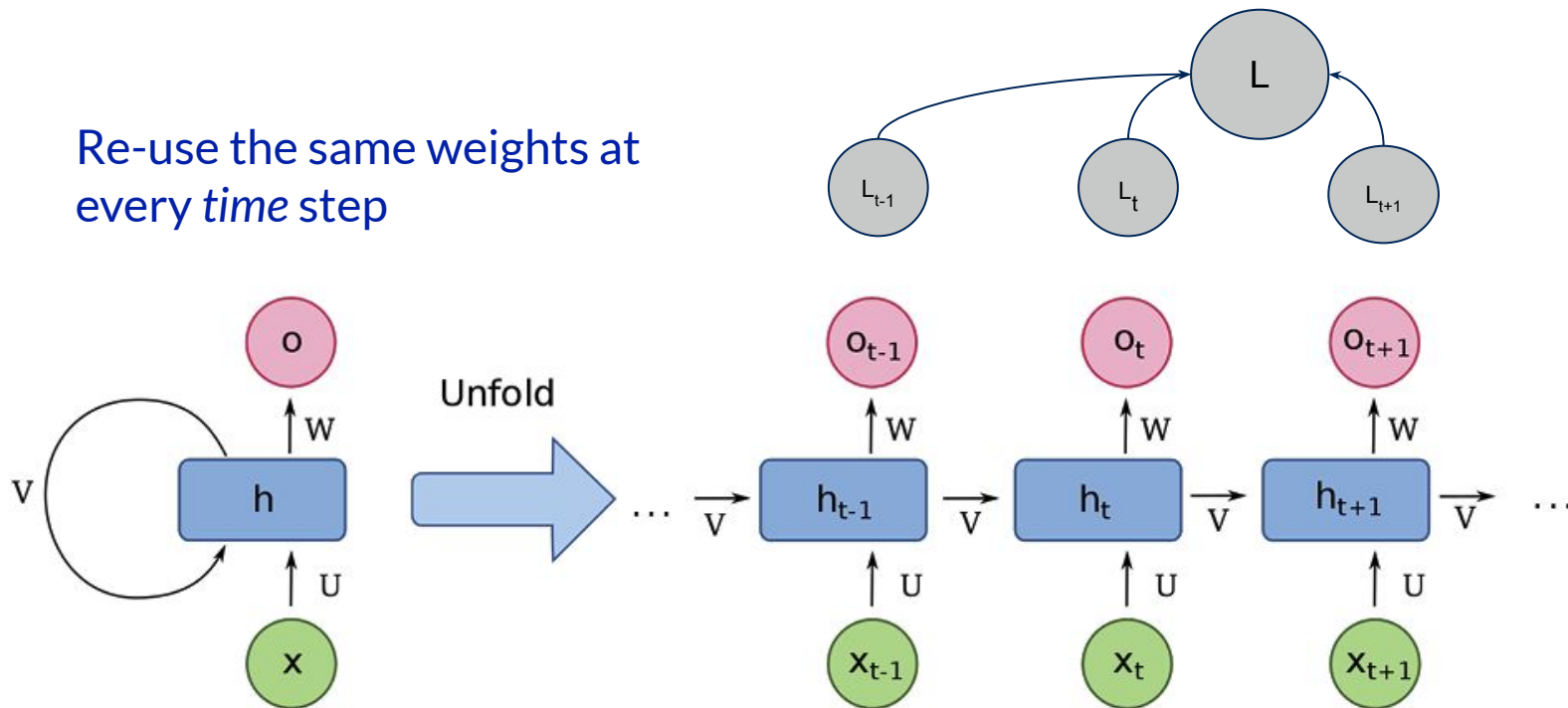
- An RNN is a function f_{θ} of type $(x_t, h_t) \mapsto (y_t, h_{t+1})$
- In words, it is a neural network that maps an input x_t into an output y_t , with the hidden vector h_t playing the role of "memory"

- U, V and W are the weights of the network
- X is the input vector
- h is the hidden vector
- O or θ are the neural network parameters



RNNs unfolding

Re-use the same weights at every *time step*



Recurrent Neurons Update

A scratch implementation is included in the Notebook RNN_Foundations pushed to our repository

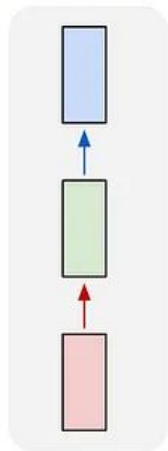


```
from torch.nn import RNN
model = RNN(input_size, rnn_units)
```

RNNs for Sequence Modelling

Vanilla RNN

one to one



Binary
classification

one to many

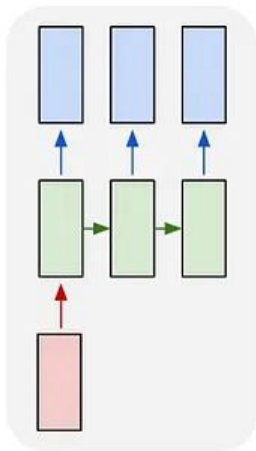
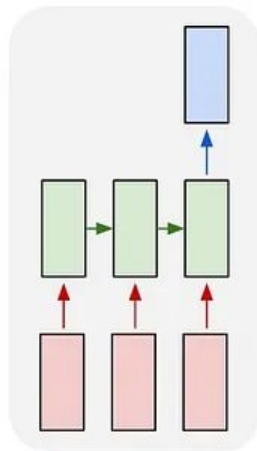


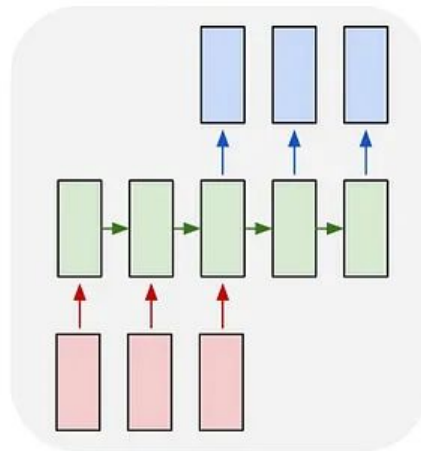
Image
Captioning

many to one



Sentiment
Analysis

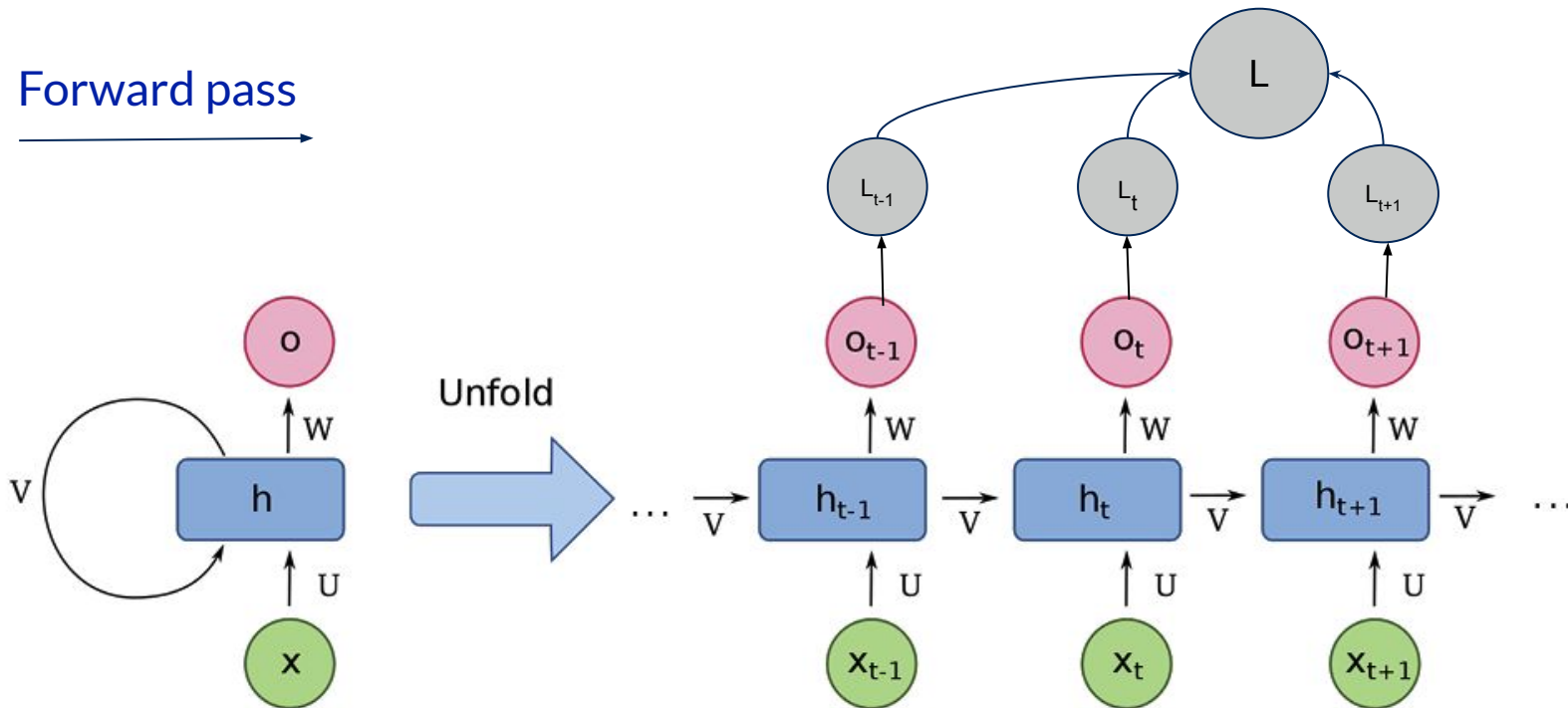
many to many



Machine Translation

Backpropagation Through Time

Forward pass

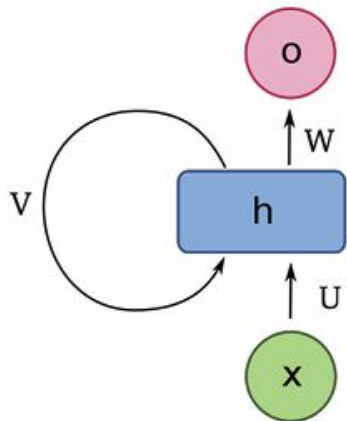


Backpropagation Through Time

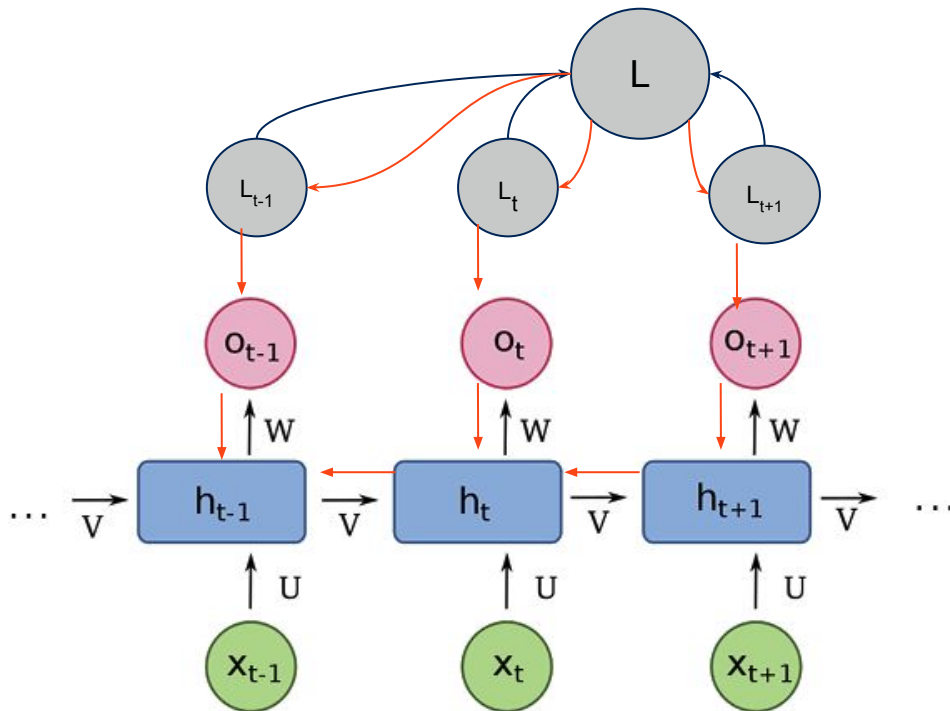
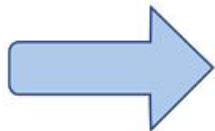
Forward pass



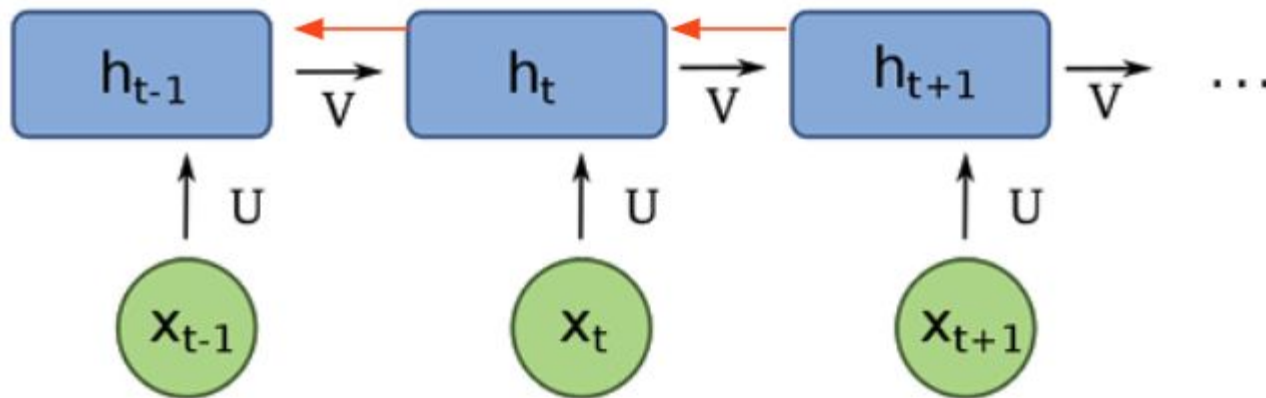
Backward pass



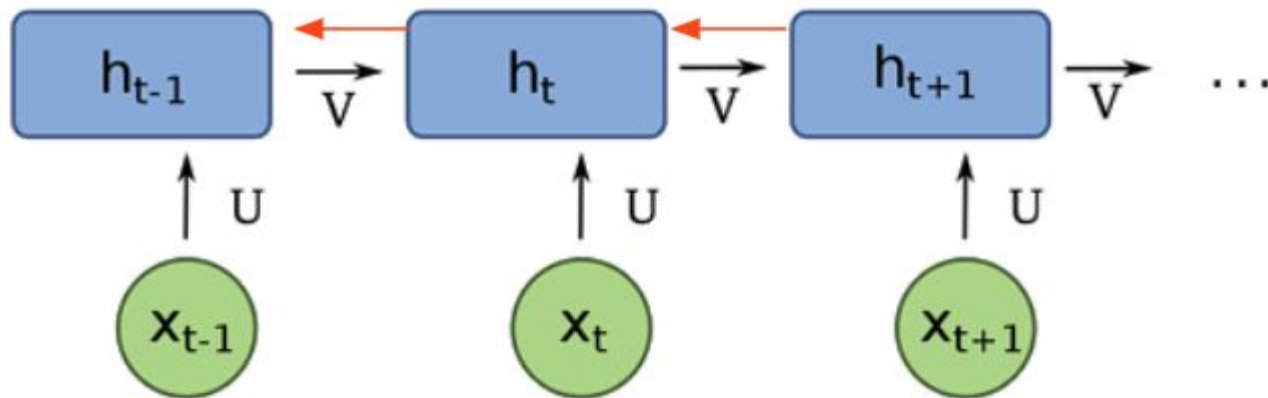
Unfold



Standard RNN Gradient Flow



Standard RNN Gradient Flow



The problem of vanishing gradients and long-term dependencies

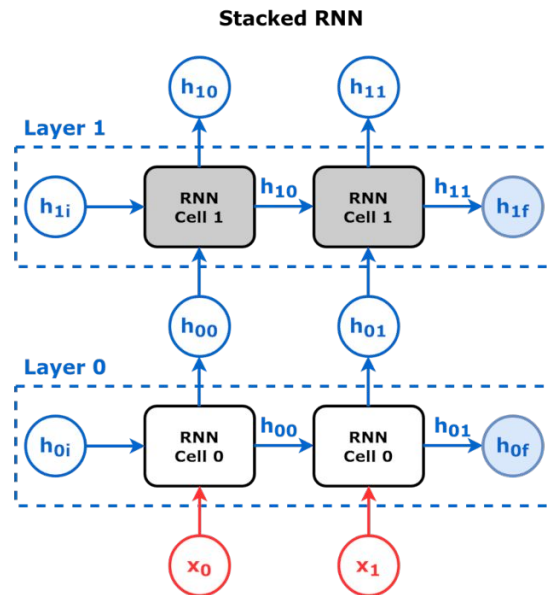
Stacked RNN

- A stacked RNN, or deep RNN, is composed of multiple RNNs stacked one above the other. Abstractly, it is structured as follows

- Layer 1 has a hidden vector $h_{1,t}$, parameters θ_1 and maps $f_{\theta_1} : (x_{0,t}, h_{1,t}) \mapsto (x_{1,t}, h_{1,t+1})$
- Layer 2 has a hidden vector $h_{2,t}$, parameters θ_2 and maps $f_{\theta_2} : (x_{1,t}, h_{2,t}) \mapsto (x_{2,t}, h_{2,t+1})$

Layer n has a hidden vector $h_{n,t}$, parameters θ_n and maps

$$f_{\theta_n} : (x_{n-1,t}, h_{n,t}) \mapsto (x_{n,t}, h_{n,t+1})$$



Wikimedia Commons

The Vanishing Gradient Problem

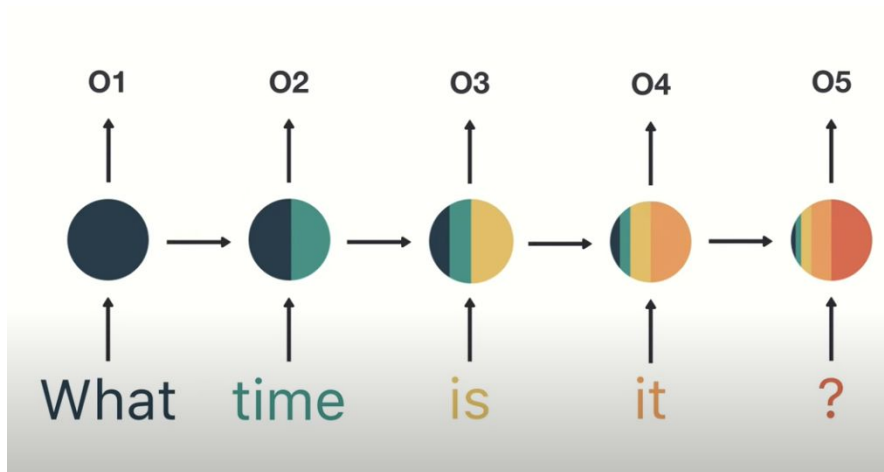
- Gradients shrink exponentially over long sequences.
- Earlier timesteps have less influence on the final output.
- Hard for RNNs to remember long-term dependencies.



As an RNN processes more information it has trouble retaining information from the early steps (**Short memory**)

Example

- Let's take a look at the sequence of words asking a question



Training Process in Recurrent Neural Networks

1. **Input at Each Time Step:** A single time step of the input sequence is provided to the network.
2. **Calculate Hidden State:** Using the current input and the previous hidden state, the network calculates the current hidden state h_t .
3. **State Transition:** The current hidden state h_t then becomes h_{t-1} for the next time step.
4. **Sequential Processing:** This process continues across all time steps to accumulate information from previous states.
5. **Output Generation and Error Calculation:** The final hidden state is used to compute the network's output, which is then compared to the actual target output to generate an error.
6. **Backpropagation Through Time (BPTT):** This error is backpropagated through each time step to update weights and train the RNN.

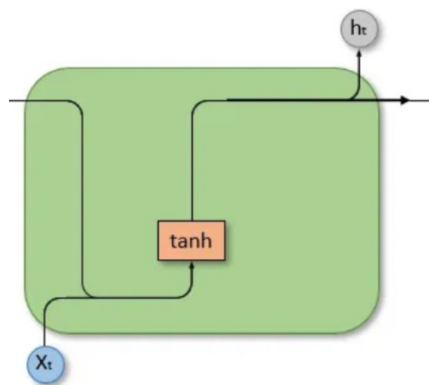


Transition to Advanced Variants

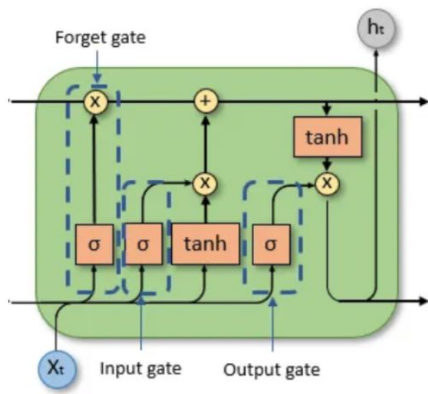
Introduction to LSTMs and GRUs

- LSTMs (Long Short-Term Memory) solve vanishing gradient issues.
- GRUs (Gated Recurrent Units) simplify LSTMs while keeping performance.
- Both use **gating** mechanisms to control memory.

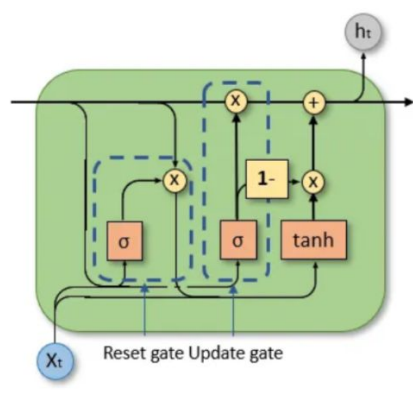
RNN



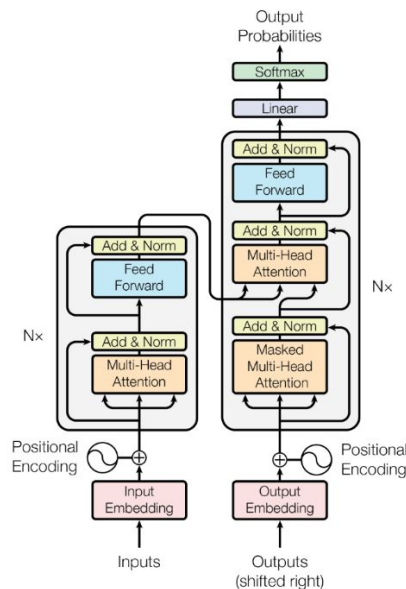
LSTM



GRU



Transformers

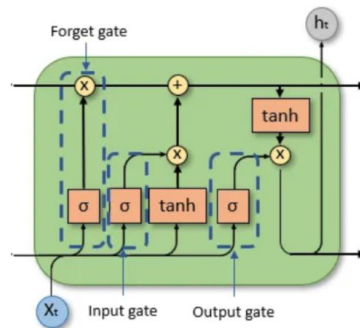


<https://aiml.com/compare-the-different-sequence-models-rnn-lstm-gru-and-transformers/>

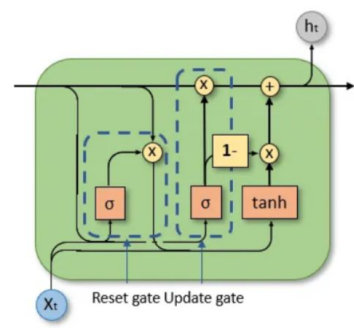
How Gating Mechanisms Work

- Gates are tensor operations that can learn what information to add or remove to the hidden state
- **Forget Gate:** Decides what information to remove.
- **Input Gate:** Determines what new information to store.
- **Output Gate:** Regulates what information should be output at the current step. This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical.

LSTM



GRU



Gated Recurrent Units (GRUs) simplify LSTMs by combining the input and forget gates into a single update gate and streamlining the output mechanism

LSTM and GRU

```
from torch.nn import LSTM
```

```
model = LSTM(input_size, rnn_units)
```

```
from torch.nn import GRU
```

```
model = GRU(input_size, rnn_units)
```

Limitations of RNNs

- They don't allow parallelization
- Even with LSTM and GRUs they don't allow long memory in general



Hands-on Implementation
